

MZ2POL-99: Best Practice Summary

Series: MZ2POL — Management Zone to Policy Migration | Notebook: 99 | Created: March 2026 | Last Updated: 04/27/2026

Overview

This notebook consolidates every actionable best practice from the MZ2POL series (notebooks 00-08) into a single definitive reference. Each practice includes the exact setting or value to use, a priority level, and its category. Use this as a pre-flight checklist before, during, and after your migration from Management Zones to Policies, Boundaries, and Segments.

Table of Contents

- 1. [Assessment and Planning](#)
- 2. [Security Context](#)
- 3. [Policy Design](#)
- 4. [Boundary Design](#)
- 5. [Segment Design](#)
- 6. [Bucket Strategy](#)
- 7. [Group and SAML Structure](#)
- 8. [Templated Policies at Scale](#)
- 9. [Migration Execution](#)
- 10. [Validation and Ongoing Maintenance](#)

1. Assessment and Planning

Practice	Recommended Setting/Value	Priority
Classify every MZ by type before migration	Categories: Team/Ownership, Environment, Geographic, Application, Infrastructure	Critical
Prioritize migration by impact	P1: Security/compliance MZs, P2: Team-based MZs, P3: Filtering-only MZs, P4: Informational/legacy MZs	Critical
Audit entity security context coverage before starting	Run <code>countIf(isNotNull(dt.security_context))</code> on services, hosts, and process groups; target 100%	Critical
Map each MZ to its replacement construct	Team MZ -> Security Context + Policy + Boundary; Environment MZ -> Boundary; Region MZ -> Segment; Application MZ -> Segment	Critical

Practice	Recommended Setting/Value	Priority
Document all MZ dependencies before migration	Inventory alerting profiles, dashboards, API integrations, SLOs, and automation workflows referencing each MZ	Critical
Use MZ2POL-00 SDK tool for MZ inventory	Run the TypeScript analysis tool against Settings API <code>builtin:management-zones</code> to export all rules in flat table format	Recommended
Identify tag patterns for segment creation	Query <code>fetch dt.entity.host \ expand tag = tags \ parse tag, "LD:tagKey ':' LD:tagValue"</code> to discover common tag keys	Recommended
Perform risk assessment per MZ	Score each MZ on likelihood and impact for: users lose access, wrong data visible, alert routing breaks, dashboard filters fail	Recommended

2. Security Context

Practice	Recommended Setting/Value	Priority
Set <code>dt.security_context</code> on every entity before creating boundaries	Assign via Entity Enrichment rules (preferred), OneAgent host properties, or OpenPipeline processing	Critical
Use a consistent naming convention for security context values	<code>team-{name}</code> for teams, <code>env-{name}</code> for environments, <code>region-{code}</code> for regions, <code>app-{name}</code> for applications	Critical
Derive security context from existing tags	Map <code>tag team:frontend</code> to <code>dt.security_context = "team-frontend"</code> to avoid double-entry	Recommended
Achieve 100% security context coverage before cutover	Any entity without <code>dt.security_context</code> becomes invisible to boundary-scoped users	Critical
Use combined naming for multi-dimensional scoping	Format: <code>{env}-{team}</code> (e.g., <code>prod-frontend</code>) when access requires both environment and team dimensions	Recommended
Automate security context for new entities	Configure auto-tagging rules or Entity Enrichment so newly deployed services/hosts get security context automatically	Critical
Leverage primary Grail fields for context	Use <code>k8s.cluster.name</code> , <code>k8s.namespace.name</code> , <code>aws.account.id</code> , <code>dt.host_group.id</code> as security context sources for Kubernetes/cloud workloads	Recommended

3. Policy Design

Practice	Recommended Setting/Value	Priority
Start with default policies; customize only when needed	<code>Dynatrace Viewer</code> for read-only, <code>Dynatrace Standard User</code> for regular users, <code>Dynatrace Professional User</code> for power users, <code>Dynatrace Admin User</code> for admins	Critical
Pair a Dynatrace access policy with a data access policy	Viewers -> <code>Dynatrace Viewer + Data Viewer</code> ; Developers -> <code>Dynatrace Standard + Data Viewer</code> ; SRE -> <code>Dynatrace Professional + Data Editor</code> ; Admins -> <code>Dynatrace Admin + Data Editor</code>	Critical

Practice	Recommended Setting/Value	Priority
Apply least privilege	Grant the minimum permissions required for each role; never use <code>ALLOW storage::*</code> without a boundary or condition	Critical
Use policy conditions with <code>WHERE</code> for scoped access	<code>ALLOW storage:logs:read WHERE storage:dt.security_context = "team-frontend"</code>	Recommended
Never create one policy per user	Assign policies to groups; bind users to groups	Critical
Never duplicate default policies	Extend with custom policies only for permissions default policies do not cover	Recommended
Use descriptive policy names	Format: <code>"Frontend Team – Standard Access"</code> not <code>"Policy 1"</code>	Recommended
Document every custom policy's purpose	Include description field when creating the policy via UI or API	Recommended

4. Boundary Design

Practice	Recommended Setting/Value	Priority
Pair Gen2 + Gen3 policies with their own boundaries	Two parallel bindings on the group: (Gen3) <code>storage:dt.security_context + settings:dt.security_context</code> for Grail data and Settings 2.0; (Gen2 transitional) <code>environment:management-zone</code> for Classic entity access. Don't bundle conditions in one boundary — each boundary only filters its own policy domain	Critical
Keep boundary conditions simple	One condition per line; each line is OR-combined within the boundary	Critical
Stay within the 10-line limit per boundary	Create multiple boundaries if you need more than 10 conditions	Critical
Use <code>startsWith</code> for environment/region prefixes	<code>storage:dt.security_context</code> <code>startsWith</code> <code>"prod-"</code> to match all production contexts	Recommended
Use <code>IN</code> for multi-value restrictions	<code>storage:dt.security_context</code> <code>IN</code> <code>("team-infra", "team-sre", "team-platform")</code> for platform teams boundary	Recommended
Create reusable boundaries	One boundary should serve multiple policy bindings; avoid one-off boundaries	Recommended
Use descriptive boundary names	Format: <code>"Production Environment"</code> , <code>"LOB5-Scope"</code> , <code>"Frontend Team Scope"</code>	Recommended
Never omit the <code>settings:dt.security_context</code> domain	Without it, settings objects remain unrestricted even when storage is scoped	Critical

Practice	Recommended Setting/Value	Priority
Never omit the <code>storage:dt.security_context</code> domain	Without it, Grail data (logs, spans, metrics) remains unrestricted even when classic entities are scoped	Critical

5. Segment Design

Practice	Recommended Setting/Value	Priority
Use segments for data filtering; use policies+boundaries for access control	Segments replace MZ filtering; policies+boundaries replace MZ permissions. Never conflate the two.	Critical
Align segments with business structure	Create segments per team, product, region, or environment — not per technical component	Critical
Use <code>matchesValue(tags, "key:value")</code> for tag-based segment filters	Replaces MZ rule <code>host tag equals value</code>	Critical
Use <code>matchesPhrase(dt.entity.service.name, "text")</code> for service name filters	Replaces MZ rule <code>service name contains</code>	Recommended
Use variables for dynamic segments	Entity variable: <code>filter dt.entity.kubernetes_cluster == \$cluster</code> ; List variable: <code>filter matchesValue(tags, concat("env:", \$environment))</code>	Recommended
Test segment filter logic with DQL before creating the segment	Run the filter as a standalone DQL query and verify results match expected entity set	Critical
Use consistent segment naming	<code>"Production Environment"</code> , <code>"Frontend Team Services"</code> , <code>"North America Region"</code> — not <code>"Segment 1"</code>	Recommended
Share segments appropriately	Private for personal use, shared for team use, public for org-wide use	Recommended
Use indexed fields in segment filters	Tags and entity IDs are indexed; avoid regex on large datasets	Recommended
Layer multiple segments for precise filtering	Select multiple segments in the app header for multi-dimensional filtering (e.g., team + environment)	Optional

6. Bucket Strategy

Practice	Recommended Setting/Value	Priority
Use buckets for physical data isolation (compliance, strict team separation)	Create team-specific or compliance buckets: <code>pci_logs</code> , <code>frontend_logs</code> , <code>prod_spans</code>	Critical

Practice	Recommended Setting/Value	Priority
Use segments (not buckets) for ad-hoc or regional filtering	Buckets are irreversible; segments are flexible. Only use buckets when physical isolation is required.	Critical
Plan bucket names carefully — they are immutable	Naming convention: <code>{team}_{datatype}</code> or <code>{env}_{datatype}</code> . 3-100 chars, lowercase alphanumeric, underscores, hyphens only.	Critical
One data type per bucket	Logs, metrics, events, or spans — never mixed in a single bucket	Critical
Stay within 80 buckets per environment	Default platform limit	Critical
Target ~1 TB/day per bucket for optimal query performance	Acceptable: 1-3 TB/day (limited query window). Hard limit: 3 TB/day per bucket.	Recommended
Create buckets and configure OpenPipeline routing BEFORE migrating policies	Data must flow to correct buckets first; policies reference bucket names	Critical
Reference buckets in both policies and boundaries	Policy: <code>ALLOW storage:logs:read WHERE storage:bucket.name = "frontend_logs"</code> . Boundary: <code>storage:bucket.name IN ("frontend_logs", "frontend_spans")</code>	Recommended
Do not use buckets for regional or application MZs	Use segments instead — buckets cannot be consolidated or split later	Recommended

7. Group and SAML Structure

Practice	Recommended Setting/Value	Priority
Use SAML groups tied to Active Directory for team management	Map AD groups to Dynatrace groups via SAML assertion; no separate user management needed	Critical
Structure: Group -> Policy + Boundary	Group: "LOB5-Team" (SAML) -> Policy: Dynatrace Professional + Boundary: LOB5-Scope (three-domain)	Critical
For stricter isolation, apply boundary to all policies on a group	Both Dynatrace Viewer and Dynatrace Professional get the same boundary	Recommended
Create one group per MZ-equivalent scope	<code>frontend-team</code> , <code>prod-viewers</code> , <code>sre-team</code> — one group per access boundary	Critical
Never assign permissions to individual users	Always use group-based policy bindings	Critical
Document group-to-MZ mapping during migration	Maintain a table: MZ name -> new group name -> policy -> boundary	Recommended

8. Templated Policies at Scale

Practice	Recommended Setting/Value	Priority
Use <code>\${bindParam:...}</code> templated policies instead of one policy per MZ	3-5 templates replace 92+ individual policies. Template: <code>ALLOW storage:*:read WHERE storage:dt.security_context = "\${bindParam:team}"</code>	Critical
Create templates by MZ type, not by MZ instance	<code>tpl-team-data-reader</code> , <code>tpl-team-data-editor</code> , <code>tpl-region-reader</code> — one template per access pattern	Critical
Use Pattern A (read-only) for viewer MZ replacements	Template grants <code>storage:logs:read</code> , <code>storage:spans:read</code> , <code>storage:metrics:read</code> scoped by <code>\${bindParam:team}</code>	Recommended
Use Pattern B (full access) for editor MZ replacements	Template grants <code>storage:*:read</code> , <code>storage:*:write</code> , <code>settings:objects:*</code> scoped by <code>\${bindParam:team}</code>	Recommended
Use multiple bindings of the same template for multi-MZ users	Bind template twice to same group with different parameter values (e.g., <code>team: checkout</code> and <code>team: shared-services</code>); bindings are additive	Recommended
Use <code>startsWith</code> in templates for region/environment patterns	<code>WHERE storage:dt.security_context startsWith "\${bindParam:region-prefix}"</code>	Optional
Automate bulk binding via IAM API	POST <code>/iam/v1/repo/account/{ACCOUNT}/bindings/{POLICY_UUID}/{GROUP_UUID}</code> with <code>{"parameters": {"team": "value"}}</code>	Recommended
Verify all bindings after bulk creation	GET <code>/iam/v1/repo/account/{ACCOUNT}/bindings/{POLICY_UUID}</code> and validate HTTP 201/409 responses	Critical

9. Migration Execution

Practice	Recommended Setting/Value	Priority
Follow the 7-phase migration order	Phase 1: Foundation -> Phase 2: Security Context -> Phase 3: Policies/Boundaries -> Phase 4: Segments -> Phase 5: Parallel Running -> Phase 6: Cutover -> Phase 7: Cleanup	Critical
Run parallel (MZ + new model) for 2-4 weeks before cutover	Both systems active simultaneously; users should see identical data via both paths	Critical
Compare entity counts between MZ and segment during parallel running	<code>countIf(in(managementZones, {"MZ-Name"}))</code> vs <code>countIf(matchesValue(tags, "key:value"))</code> — counts must match	Critical

Practice	Recommended Setting/Value	Priority
Update dashboards to use segments before cutover	Replace MZ filters with segment selectors at dashboard level and tile level	Critical
Update alerting profiles to use segments before removing MZs	Alerts scoped to MZs will break when MZs are removed	Critical
Update API integrations to use new access model before cutover	API calls with MZ parameters must be converted to policy-based authentication	Recommended
Archive MZ configurations before deletion	Export via Settings API and store in version control	Recommended
Wait 2+ weeks after cutover before deleting MZs	Verify no remaining MZ dependencies in alerting, dashboards, or API integrations	Critical
Send cutover communication to all users	Include: what changed, action required (select segments in apps), documentation link, support contact	Recommended
Prepare a rollback plan	Document how to re-enable RBAC MZ assignments if critical access issues arise post-cutover	Critical

10. Validation and Ongoing Maintenance

Practice	Recommended Setting/Value	Priority
Validate security context coverage reaches 100%	<code>fetch dt.entity.service \ summarize total = count(), covered = countIf(isNotNull(dt.security_context))</code>	Critical
Compare MZ membership vs security context alignment per MZ	Query entities <code>in(managementZones, {"MZ"}) AND dt.security_context == "expected-value"</code> — all must match	Critical
Find entities in MZ but missing from segment	<code>filter in(managementZones, {"MZ"}) \ filter NOT matchesValue(tags, "key:value")</code> — result must be empty	Critical
Test each user type with a test account	Log in as test user, verify expected data visible, verify restricted data hidden	Critical
Audit security context weekly	Run coverage query to catch new entities deployed without security context	Recommended
Review segment effectiveness monthly	Verify segments return expected entity counts and data	Recommended
Perform access review quarterly	Verify user group memberships, policy bindings, and boundary conditions are still appropriate	Recommended
Monitor for untagged entities	<code>fetch dt.entity.service \ filter isNull(dt.security_context) \ filter NOT matchesValue(tags, "team:*")</code> — track count on a dashboard	Recommended
Build an access health dashboard	Include KPIs: security context coverage %, entities per context, untagged entity count	Optional

Practice	Recommended Setting/Value	Priority
Automate onboarding for new entities	Auto-tagging + Entity Enrichment rules ensure new services/hosts inherit correct security context	Recommended

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.